

GEWN — System Architecture

Project GEWN | AI-Native Desktop Copilot Platform

Document Type: System Architecture Specification

Audience: Engineers, Technical Collaborators, System Architects

Status: Living Document — v1.1

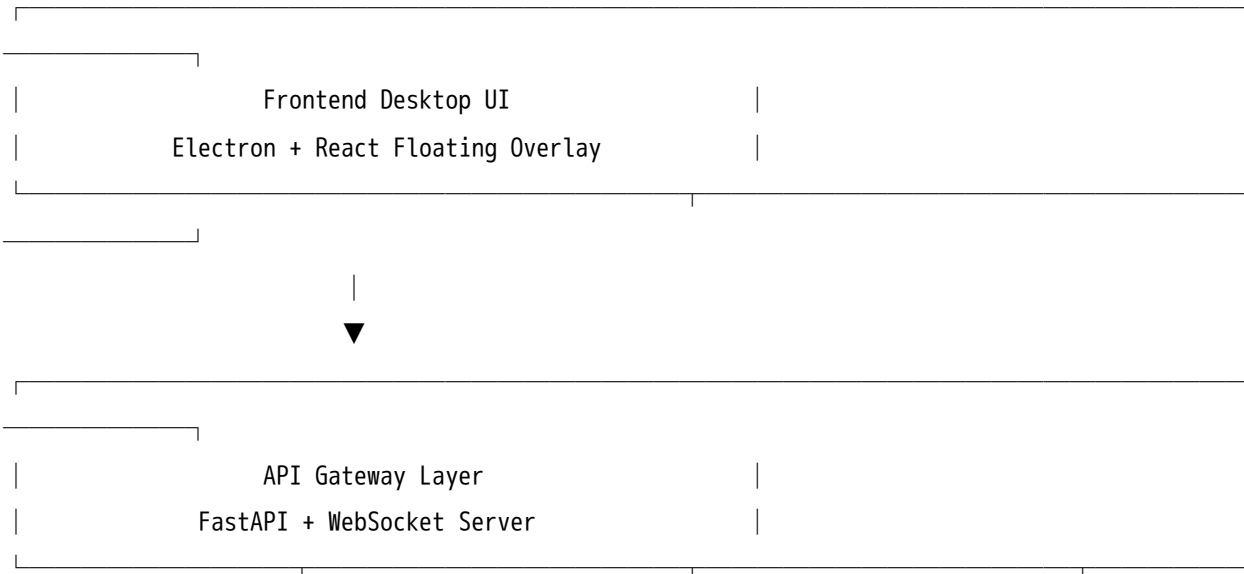
Overview

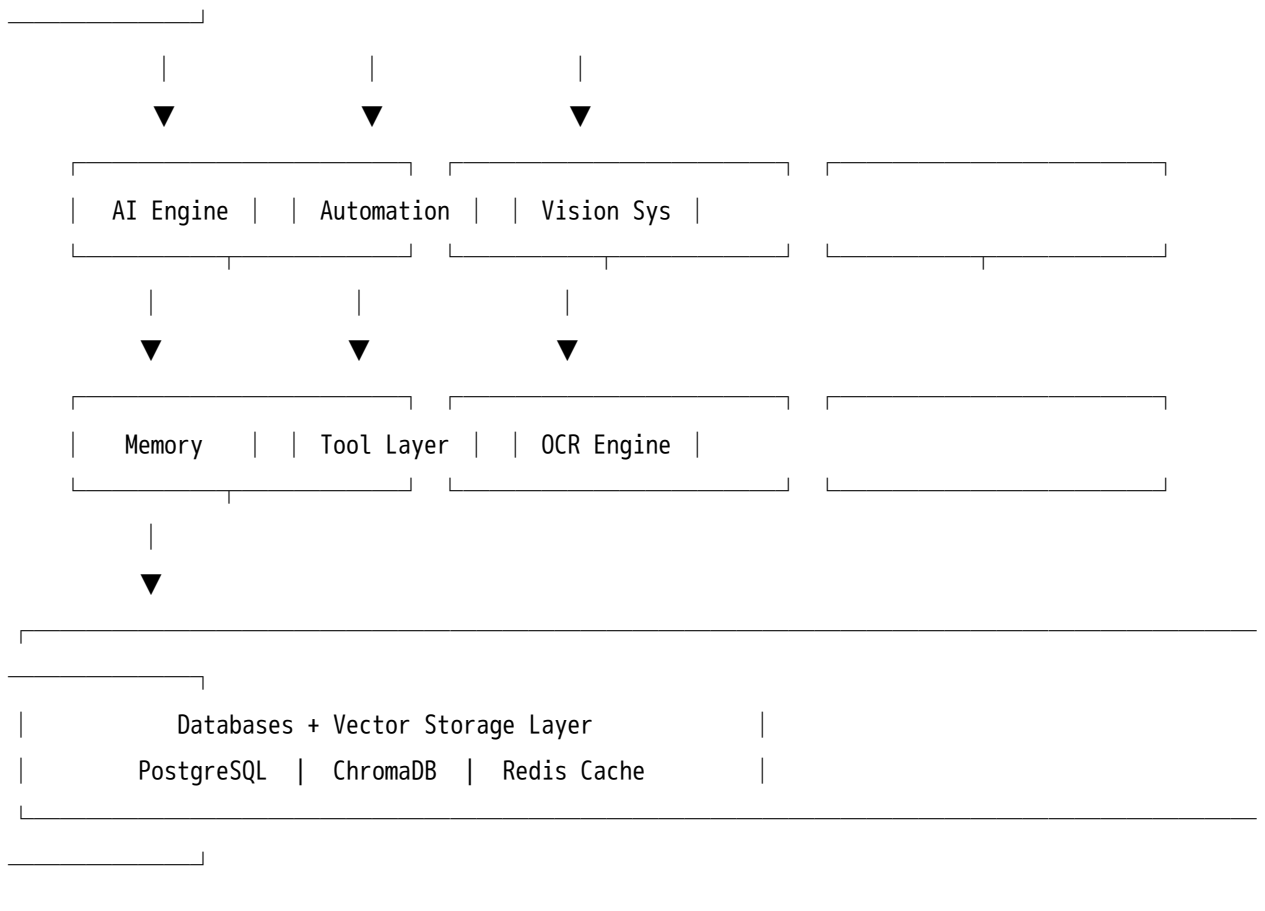
GEWN is a modular AI operating companion built around five core pillars: **conversational intelligence**, **desktop automation**, **multimodal screen understanding**, **semantic memory**, and **autonomous task orchestration**.

The architecture follows a layered, service-oriented design to maximize:

- **Modularity** — each subsystem is independently operable and replaceable
- **Real-time responsiveness** — streaming-first design across all interaction surfaces
- **Scalability** — local-first with clean cloud expansion paths
- **Extensibility** — plugin architecture and open SDK for third-party agents

1. High-Level System Design





2. Frontend Layer

Role: Visual interaction surface between the user and GEWN.

Stack: Electron · React · Tailwind CSS · Framer Motion

Key Modules

Floating Overlay System Always-on-top, transparent, draggable assistant window. Renders over any application without disrupting active workflows. Supports click-through passthrough when idle.

Chat Interface Streaming AI response rendering with full Markdown support, syntax-highlighted code blocks, and inline tool result display. Designed for conversational density without visual clutter.

Voice UI Real-time audio waveform visualization, wake word status indicator, and push-to-talk controls. Provides clear feedback on STT/TTS pipeline state.

Overlay Guidance Engine Renders context-sensitive annotations, step-by-step workflow guides, and error highlights directly on the user's screen. Uses a transparent Electron window layer with SVG annotations.

3. Backend Layer

Role: Orchestration core for all AI operations, system integrations, and workflow execution.

Stack: FastAPI · Python · Redis · Docker

Components

API Gateway Central entry point handling authentication, request routing, and response streaming. All client-server communication flows through this layer, including REST endpoints and persistent WebSocket connections.

Workflow Orchestrator Coordinates the full task execution lifecycle: receives user intent → dispatches to the appropriate AI agent → retrieves memory context → routes to tool execution → streams results back. Acts as the central nervous system of GEWN.

WebSocket Layer Manages real-time bidirectional channels for:

- Live chat streaming (token-by-token)
- Voice audio streaming (PCM → STT → response → TTS)
- Screen event updates (window change, OCR trigger)

4. AI Engine

Role: Reasoning, planning, and intelligence layer.

Stack: LangChain · LangGraph · OpenAI APIs · Google Gemini APIs

Components

Conversational Engine Handles natural language understanding, response generation, summarization, and contextual reasoning. Supports multi-turn dialogue with injected context from

memory and screen state.

Agent System Specialized agents with distinct tool sets and prompt strategies:

Agent	Primary Capability
Coding Agent	Code generation, debugging, explanation, LSP integration
Research Agent	Web search, PDF summarization, knowledge synthesis
Automation Agent	File system, app control, browser automation
Memory Agent	Context storage, retrieval, and consolidation
Scheduling Agent	Calendar management, reminders, recurring task queues

Planning Engine Decomposes high-level user goals into executable subtask graphs. Manages tool selection, ordering, retry logic, and human-in-the-loop confirmation gates for irreversible actions.

5. Memory System

Role: Stores and retrieves contextual knowledge to power personalization and continuity.

Stack: ChromaDB / Pinecone · PostgreSQL · Redis · OpenAI Embeddings

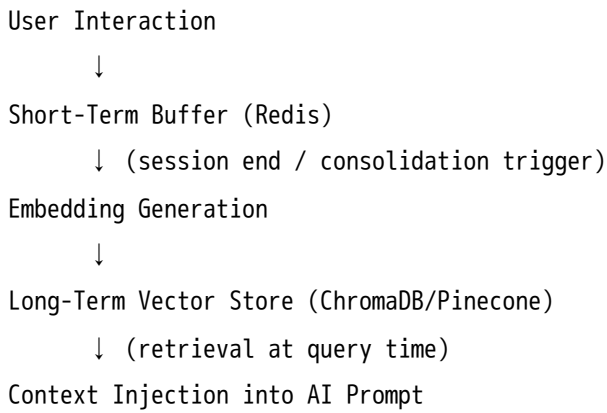
Memory Layers

Short-Term Memory (Redis) Active session context: current conversation thread, in-progress task state, recent tool outputs. TTL-managed with automatic expiry. Enables coherent multi-turn interactions within a session.

Long-Term Semantic Memory (Vector DB) Persistent storage of user preferences, project history, past workflows, and interaction summaries. Embedding-based retrieval allows fuzzy, semantic recall rather than exact lookup.

Structured Metadata (PostgreSQL) User accounts, settings, workflow definitions, plugin configurations, and action audit logs. Source of truth for structured, relational data.

Memory Flow



6. Tool Execution System

Role: Enables GEWN to take direct action on the operating system and web.

Stack: PyAutoGUI · Playwright · Python stdlib

Tool Categories

Filesystem Tools Create, read, rename, move, delete files and directories. All destructive operations pass through the Permission Layer before execution. Supports dry-run preview mode.

Application Tools Launch, focus, switch, and close applications. Reads window state via OS accessibility APIs (Windows COM, macOS AppleScript/JXA). Powers workflow automation and smart workspace setup.

Browser Tools Full Playwright-driven browser automation: navigation, DOM interaction, form filling, content extraction, and screenshot capture. Supports headless and headed modes.

System Tools Terminal command execution (sandboxed), clipboard read/write, notification dispatch, and system state queries (CPU, disk, active processes).

7. Voice Pipeline

Role: Real-time voice interaction from wake word to spoken response.

Stack: Porcupine · Whisper / Deepgram · ElevenLabs / Coqui TTS

Pipeline Flow

Microphone Input



Wake Word Detection (Porcupine – on-device)



Speech-to-Text (Whisper API / local Whisper.cpp)



Intent Parsing + AI Processing (AI Engine)



Response Generation (Streaming text)



Text-to-Speech Synthesis (ElevenLabs / Coqui)



Audio Output

Design Notes

- Wake word detection runs entirely on-device with negligible CPU overhead
 - STT supports both cloud (Whisper API, Deepgram) and local (Whisper.cpp) backends — toggled by privacy mode
 - TTS synthesis begins streaming before full response is generated, minimizing perceived latency
 - Voice Activity Detection (VAD) handles turn-taking and barge-in interruption
-

8. Vision Module

Role: Screen awareness and multimodal understanding.

Stack: PaddleOCR · Tesseract · Multimodal LLM APIs (GPT-4 Vision / Gemini Vision)

Components

Screenshot Pipeline Periodic or event-triggered screen capture. Differential analysis detects meaningful content changes to avoid redundant processing. Region-of-interest cropping reduces token overhead for multimodal LLM calls.

OCR Engine Extracts machine-readable text from any visible screen region — UI labels, code editors, browser pages, terminal output. PaddleOCR as primary engine; Tesseract as fallback for structured documents.

Screen Understanding Multimodal LLM analysis of captured screenshots for semantic comprehension: error detection, layout analysis, workflow state inference. Results injected into the AI context window.

Overlay Renderer Electron transparent window positioned over the screen. Renders SVG-based annotations, step indicators, and error highlights targeting specific screen coordinates.

9. Database Architecture

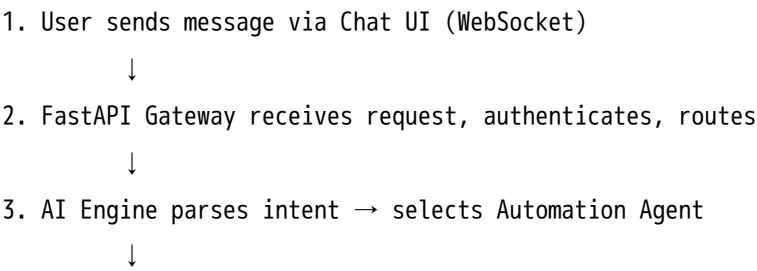
Layer	Technology	Stores
Relational	PostgreSQL	Users, settings, workflow definitions, audit logs
Vector Store	ChromaDB / Pinecone	Embeddings, semantic memory, document chunks
Cache	Redis	Session state, real-time context, short-term memory

Design Principles

- PostgreSQL is the authoritative source for all structured state
- Vector stores are derived/indexed data — can be rebuilt from source if needed
- Redis is ephemeral — no critical state lives only in cache
- All databases are containerized via Docker for local development parity

10. API Request Lifecycle

Example: *"Organize my downloads folder."*



4. Memory System retrieves relevant context
(past folder preferences, file type rules)
↓
5. Planning Engine decomposes task into subtasks:
[scan folder] → [classify files] → [create subfolders] → [move files]
↓
6. Permission Layer: confirm destructive moves with user
↓
7. Tool Execution System runs filesystem operations
↓
8. Results streamed back to Frontend via WebSocket
↓
9. Memory System stores interaction summary
-

11. Local vs. Cloud Processing

GEWN is designed **local-first** with cloud augmentation for high-complexity tasks.

Concern	Local Processing	Cloud Processing
Workloads	File ops, desktop automation, OCR, wake word, session cache	LLM inference, advanced reasoning, large-scale embeddings
Latency	Low (sub-50ms for most ops)	Variable (200ms–2s for LLM calls)
Privacy	Full — no data egress	Data sent to provider APIs
Availability	Works offline	Requires internet
Performance	CPU-bound, hardware-constrained	Scales with provider compute

Privacy Mode routes all AI processing to local models (Ollama + Llama 3 / Mistral / Phi-3), keeping all data on-device.

12. Security Architecture

Permission Layer

Every action is classified by risk level before execution:

Risk Level	Examples	Behavior
Low	Read file, open app, search web	Auto-execute
Medium	Write file, send message, API call	Log + execute
High	Delete files, terminal commands, OAuth flows	Explicit user confirmation required

Encryption

- User memory and preferences: AES-256 at rest
- API keys and credentials: OS keychain integration (Keychain on macOS, Credential Manager on Windows)
- Transport: TLS 1.3 for all cloud API communication

Sandbox Execution

- Browser automation runs in isolated Playwright contexts
- Plugin code executes in a restricted Python sandbox with limited OS access
- Terminal commands require explicit allowlist configuration

Privacy Mode

Full local execution path using Ollama. No telemetry, no cloud API calls, no data egress. Toggled per-session or set as default in user preferences.

13. Plugin Architecture

Purpose: Allow third-party developers to extend GEWN with custom agents, commands, workflows, and integrations.

Plugin Structure

```
plugin/
├── manifest.json      # Name, version, permissions, entry points
├── commands/         # Reusable command definitions
└── workflows/        # Workflow templates
```

└── agents/ # Custom AI agent implementations
└── integrations/ # External API connectors

Plugin Lifecycle

- 1. **Discovery** — Plugin registry scans plugins/ directory at startup
- 2. **Validation** — Manifest schema validation + permission audit
- 3. **Registration** — Commands/agents registered in the core routing table
- 4. **Execution** — Plugin code invoked via event hooks with scoped context
- 5. **Isolation** — Plugins cannot access core memory or system tools beyond declared permissions

Event Hooks

- on:message — Fires on every user message
- on:screen-change — Fires when active window or screen content changes
- on:task-complete — Fires when an agent finishes a task
- on:schedule — Fires on cron-based triggers

14. Scalability Design

Current (Monolithic Local)

All services run within a single Docker Compose stack on the user's machine. Suitable for individual use and early development.

Near-Term (Service Isolation)

Split AI Engine, Automation, Memory, Vision, and Voice into independent services communicating over internal APIs. Enables independent scaling and fault isolation.

Future (Microservice + Cloud Hybrid)

Service	Scaling Strategy
AI Engine	Horizontal — multiple LLM workers behind a load balancer

Service	Scaling Strategy
Memory	Read replicas for vector store; write-ahead log for PostgreSQL
Automation	Queue-based (Celery + Redis) for concurrent task execution
Voice	Dedicated streaming service with WebRTC for real-time audio

Cross-Device Sync (Future)

Encrypted memory and workflow sync across desktop, mobile, and cloud via a GEWN Sync service. All sync operations are end-to-end encrypted.

15. Recommended Initial Stack

Layer	Technology
Desktop UI	Electron + React + Tailwind CSS + Framer Motion
Backend	FastAPI + Python + WebSockets
AI Orchestration	LangChain + LangGraph
LLM APIs	OpenAI GPT-4o, Google Gemini 1.5 Pro
Local LLMs	Ollama (Llama 3, Mistral, Phi-3)
Vector Store	ChromaDB (local) → Pinecone (cloud)
Relational DB	PostgreSQL
Cache	Redis
Automation	PyAutoGUI + Playwright
OCR	PaddleOCR + Tesseract
Voice STT	Whisper API + Whisper.cpp (local)
Voice TTS	ElevenLabs + Coqui TTS (local)
Wake Word	Porcupine
DevOps	Docker + Docker Compose + GitHub Actions

16. Future Architecture Roadmap

Local AI Infrastructure Full offline capability using Ollama with quantized models (GGUF) on consumer hardware. GPU acceleration via CUDA/Metal for latency-sensitive workloads.

Cross-Device Synchronization Encrypted workflow and memory sync across desktop, mobile, and browser. GEWN Sync service with conflict resolution.

Enterprise AI Workspace Shared agent pools, team memory namespaces, and collaborative workflow definitions for organizational deployment.

Autonomous Workflow Runtime Persistent background execution engine for long-running tasks. Checkpoint/resume support, progress reporting, and failure recovery. Built on Celery task queues with Redis broker.

AI Operating Layer Long-term evolution: deep OS hooks, predictive context loading, and GEWN as the primary interface layer for all computing activity on the device.

Architecture Philosophy

GEWN is designed around the conviction that AI should be **contextual, proactive, multimodal, adaptive, and deeply integrated** into how people work — not bolted on as an afterthought.

Every architectural decision prioritizes:

- **Modularity** — swap any component without rebuilding the system
- **Real-time responsiveness** — streaming everywhere, blocking nowhere
- **Privacy by default** — local-first, with cloud as opt-in enhancement
- **Trustworthy automation** — explicit permissions, full audit trails, safe defaults

GEWN is not a chatbot with a desktop widget. It is an intelligent operating companion designed to evolve toward a fully AI-native computing environment.